

1 Introduction

Two galaxies passing by each other is an incredibly complex event. The gravitational pull of one galaxy on the other, and the pull of the components of the galaxies between themselves, create complex structures such as tidal streams and send both galaxies into new directions. Due to the length of the event, they are also impossible to observe in their entirety. More complex simulations also allow the rewinding of one particular event to understand what led to a known object. Similarly, due to the size and complexity of the event, analytical solution which explain the evolution of the objects accurately are also impossible. However, it is possible to create simple models which can run on any machine and are sufficiently accurate for most needs. Modelling them is thus the best way to understand how they evolve before, during and after the flyby. Galactic flyby in particular can be modelled with a simple algorithm involving only the gravitational pull from particle interactions. While this model will not reproduce small scale or particular events, it shows the broader evolution of galaxies after a flyby.

On the other hand, because it happens on a much smaller scale, the formation of a single star necessitates a more complex modellisation. Stars form when some regions in molecular clouds become dense enough and collapse on themselves. Due to the velocity of the particles falling inward, if only the gravitatal pull is taken into account, the sphere will collapse and expand in turn, indefinitely. To accurately model a star forming, other mechanisms must be taken into account. Two such mechanisms are the internal pressure and the viscosity of the collapsing gas which apply to each particle due to the neighbouring ones.

This project is composed of two parts. In the first part, we will implement a numerical model of a moving lower mass galaxy flying by a stationary higher mass galaxy. In the second part, we will attempt to model the formation of a star. Both models use N-body simulations. Due to its smaller size, the stellar model is of much higher resolution, as the same number of particle can be used to model the large galactic fly-by and the much smaller star formation.

2 Implementation

Both models were based on the same structure. A main file called forward multiple functions to initiate, evolve and plot the galaxies and stars.

The first set of functions placed N particles in a circle around a center of mass (x, y) with a normal distribution for the radius, and a uniform distribution for the angle. It calculated an initial velocity in the x and y direction using 0.4 of a Keplerian velocity, which is dependent on the radius from the center of mass. Then the acceleration of each particle was calculated from this initial velocity and the position of all the other

particles in the simulation. A second set of functions updated the positions, velocity and acceleration of the particles after a time step dt using a leapfrog algorithm. It first calculated an updated velocity after half a step using the previous acceleration, then calculated an updated position using this new velocity, before getting the updated acceleration and taking the second half-step in velocity using this new acceleration. This is the leap-frog algorithm:

```
new_v = v + dt/2 * acc
new_p = p + dt*new_v
new_acc = getAcceleration(*args)
new_new_v = new_v + dt/2*new_acc
```

Where `getAcceleration()` calculates the acceleration of the particles using the different equations discussed bellow. A last set of functions created and updated the plots, making a series of image every few time steps.

2.1 Equation of motion

The equation of motion was the most complex part of this model, as it had to be calculated individually for each particle. It was present in two different ways between the two models, and this was the largest difference between them. One of the model only included gravitational effects on the motion of the particles, while the other one included both the gravitational effects, the pressure gradient, and the effects of viscosity. The gravitational effects were a weighted sum of the pull of the other particles as a function of their mass and distance to the particle of interest:

$$\vec{F}_j = m_j G \sum_{i \neq j} \frac{m_i}{r_{ji}^3} \vec{r}_{ij} = m_j \vec{a}_j \Leftrightarrow \vec{a}_j = G \sum_{i \neq j} \frac{m_i}{r_{ji}^3} \vec{r}_{ij}.$$

The pressure gradient was slightly more complex, as it needed to be calculated as a function of the average density around the particle of interest instead of its interaction with the other individual particles. For this, we define a density kernel at each point in space which depends on the presence of other particles within the vicinity. The density at a location $\vec{r}$ is:

$$\rho(\vec{r}) = \sum_i m_i W_i(|\vec{r} - \vec{r}_i|, h).$$

Where W_i is a smoothing kernel, such that:

$$W_i(q, h) = C \begin{cases} (2 - q)^3 - 4(1 - q^3) & \text{if } 1 > q \geq 0, \\ (2 - q)^3 & \text{if } 2 > q \geq 1, \\ 0 & \text{if } q \geq 2. \end{cases} \quad (1)$$

And $C = \frac{5}{14\pi h^2}$, $q_i = \frac{|\vec{r} - \vec{r}_i|}{h}$ and h is a constant which we chose to be $0.02 \cdot R_\odot$. From this, assuming an isothermal equation of state, it is possible to obtain the pressure : $P = \kappa \rho^2$, and from it, the pressure gradient:

$$\vec{\nabla} P(\vec{r}_j) = 2\kappa \sum_i m_i \vec{\nabla}_j W(|\vec{r}_j - \vec{r}_i|, h)$$

Finally, the impact of viscosity is simply a constant ν multiplied by the speed of the particle j at time t . The exact values used in this report for the different models can be found in [Appendice A](#).

2.2 Acceleration of the code

As an N-body simulation is extremely taxing for the machine running it, multiple methods were used to accelerate the processus, and make it less taxing. There were two main possible path. The first one was to use the Numba package and compile the Python code into a language which the machine has less difficulty running. It also included the parallelisation of some parts of the code. In particular, for loops are extremely slow in Python. Running multiple of the items parallely accelerated the code significantly. The second method was to entirely erase the need for for loops by using NumPy array maths.

The code which removes the need for loops is reproduced here:

```
p_sub_0 = np.repeat(p[0,:], len(p[0,:]), axis=0).reshape(len(p[0,:]),
                                                         len(p[0,:]))
p_sub_1 = np.repeat(p[1,:], len(p[1,:]), axis=0).reshape(len(p[1,:]),
                                                         len(p[1,:]))
r_ji_x = p[0,:].transpose()-p_sub_0
r_ji_y = p[1,:].transpose()-p_sub_1

#Clip anything too close to 0
r_ji = np.clip(np.sqrt((r_ji_x)**2 + (r_ji_y)**2), 0.1*Rg, None)

#Get the acceleration
acc[0,:] = - G * sum((mp[:] / (r_ji)**3)*r_ji_x)
acc[1,:] = - G * sum((mp[:] / (r_ji)**3)*r_ji_y)
```

It creates a larger matrix of size *number of particle* $\times$ *number of particles*, such that when calculating the resulting radius, it should produce this sequence of arrays:

$$r_{ji,x} = \begin{bmatrix} p_{x1} \\ p_{x2} \\ p_{x3} \end{bmatrix} - \begin{bmatrix} p_{x1} & p_{x2} & p_{x3} \\ p_{x1} & p_{x2} & p_{x3} \\ p_{x1} & p_{x2} & p_{x3} \end{bmatrix} = \begin{bmatrix} p_{x1} - p_{x1} & p_{x1} - p_{x2} & p_{x1} - p_{x3} \\ p_{x2} - p_{x1} & p_{x2} - p_{x2} & p_{x2} - p_{x3} \\ p_{x3} - p_{x1} & p_{x3} - p_{x2} & p_{x3} - p_{x3} \end{bmatrix}$$

Then:

$$r_{ji} = \sqrt{\begin{bmatrix} p_{x1} - p_{x1} & p_{x1} - p_{x2} & p_{x1} - p_{x3} \\ p_{x2} - p_{x1} & p_{x2} - p_{x2} & p_{x2} - p_{x3} \\ p_{x3} - p_{x1} & p_{x3} - p_{x2} & p_{x3} - p_{x3} \end{bmatrix}^2 + \begin{bmatrix} p_{y1} - p_{y1} & p_{y1} - p_{y2} & p_{y1} - p_{y3} \\ p_{y2} - p_{y1} & p_{y2} - p_{y2} & p_{y2} - p_{y3} \\ p_{y3} - p_{y1} & p_{y3} - p_{y2} & p_{y3} - p_{y3} \end{bmatrix}^2}$$

Which produces an n by n matrice where each row is the radius between a particle and all the others. Finally, the calculation of the acceleration sums over the entire row, summing over the contribution of each particle i on the acceleration of particle j .

To test the comparative speed of the three methods, I used a smaller simulation, with only 500 particles in one galaxy, and 250 in the other. The three methods being:

with no acceleration, using Numba, and using array maths. The fastest was the Numba method, at 529.2 seconds, followed by the array method at 1309 seconds, more than double the time. As expected, the method with no acceleration was last, taking nearly 1500 seconds. So, while compiling the code in Numba and running multiple loops in parallel is the best option, reducing the number of for loop did significantly improve the running of the simulation. The large matrix it has to build and the plotting end up being the longer operation instead of the for loops, and they take less time.

There are probably ways in which the no loops methods could be improved, but due to time constraints, it could only be tested with basic mechanisms. It does work as a proof of concept.

3 Results

3.1 Galaxy Merger

Using the galaxy merger model, I run four different simulations. The parameters used for all of them can be found in [Appendix A](#). The first one had a Milky Way size galaxy interacting with an extremely low mass galaxy, of only 10^5 times the mass of the Sun. The second was interacting with a larger galaxy, although still low mass, of about 10^{10} the mass of the Sun. The third also included a change in mass, with two galaxies of about equal mass interacting. The last was slightly different in that it included the Milky Way size galaxy and the $10^{10} M_{\odot}$ ones, but with a head-on collision instead of the offset fly-by the others modelled.

3.2 Extremely Low Mass Galaxy

3.3 Stellar Formation

Due to a first misunderstanding as to how the smoothing kernels were supposed to work, and a lack of time to repair the code once the misunderstanding was resolved, the stellar formation model did not work. The particles stayed mostly stationary with accelerations of values around 10^{-63} , far too small compared to what they should have been.

4 Discussion

As mentioned in the [Introduction](#), numerical simulations are necessary to understand phenomena which can not be observed in their entirety and are too complex for analytical solutions. Although quite basic, the simulations in this report already represent the phenomena they model quite well.

Concerning their accuracy, the results within the galaxy merger simulation depend heavily on the initial radial velocity of the particles. Giving them too much velocity at first will make them fly away instead of rotating around the center of mass of the galaxy. This means that the galaxies will disperse before they interact, or that the interaction can be affected by this too high initial velocity. It is also an extremely

simplified model, which only include the gravitational interaction of heavy collisionless particles. More complex models show that some collision are necessary to describe accurately the evolution of galaxies. Furthermore, due to the limit imposed by the machine it is run on, the simulations have to be low resolution. Each increase in the number of particles proportionnaly increases the time it takes the simulations to run, and the power it takes to run them. Most portable computers are incapable of running more complex models. Using Numba or reducing removing for loops and plotting routines do improve this point, but neither can improve it indefinitely.

While the stellar formation model produced by with this report does not work, it is possible to make some assumptions as to what works and what could be improved. It is slightly more complex, and due to the smaller size of the object being modelled, it can be simulated in higher resolution by using the same number of particles as within the galaxy simulations. It does however make some assumptions which could affect the results. In particular, it does not take into account magnetic fields, which are known to be very strong in star formation discs and young stars. It also assumes an isothermal model, which does not take changes in temperature into account. Temperature is important in star formation, as it is one of the variables in the calculation of Jeans' mass. This mass determines whether a molecular cloud can collapse into a star or not.

Finally, both models are made in closed box simulations, where only the initial particles and objects interact. This is not an entirely accurate representation, especially for the stellar formation model. The galaxies in the galaxy merger model are large enough and far enough away from other heavy objects that they should not be overly affected. However, star form in groups with other stars, and are surrounded by many other elements. It is mentioned in the instructions for the model that the final stars are heavier in the center than they should be. This could be linked to this problem. In these closed box model, all the particles end up in the star after a certain time, instead of some of them escaping or being pulled away. Radiation pressure is also not taken into account. Particles are pushed away from each others by their emissions.

In conclusion, these simulations both are an accurate model of the astrophysical phenomena they aim to represent. However, multiple effects which have been shown to affect both the events were not included in the simulations. Thus, while they can be used to understand the movements of large structures, the minute details can not be understood with their help.

The code can be found at , along with the LAtex version of this report and the plots made during the simulations

5 Appendices

5.1 Appendice A

Common to all simulations	
M_g	$1.54 \cdot 10^{12}$
R_g	$9.5 \cdot 10^{17}$
τ	$\frac{R_\odot}{G \cdot M_g}^{0.5}$
dt	$0.01 \cdot \tau$
n_{dim}	2
n_{steps}	10 000
Simulations	
1 - Extremely Low Mass galaxy	
Galaxy 1	
N_1	1000
M_1	M_g
R_1	R_g
Offsets (position, velocity)	None
Galaxy 2	
N_2	500
M_2	$1.54 \cdot 10^5 \cdot M_\odot$
R_2	$3.8 \cdot 10^{17}$
$x_{0,2}$	$15 \cdot R_g$
$y_{0,2}$	$2.5 \cdot R_g$
$v_{x0,2}$	$\frac{-R_g}{2 \cdot \tau}$
$v_{y0,2}$	0
2 - Low Mass galaxy	
Galaxy 1	
N_1	1000
M_1	M_g
R_1	R_g
Offsets (position, velocity)	None
Galaxy 2	
N_2	500
M_2	$1.54 \cdot 10^{10} \cdot M_\odot$
R_2	$3.8 \cdot 10^{10}$
$x_{0,2}$	$15 \cdot R_g$
$y_{0,2}$	$2.5 \cdot R_g$
$v_{x0,2}$	$\frac{-R_g}{2 \cdot \tau}$
$v_{y0,2}$	0
3 - Equal Mass galaxy	
Galaxy 1	
N_1	1000
M_1	M_g
R_1	R_g
Offsets (position, velocity)	None

Galaxy 2	
N_2	1000
M_2	M_g
R_2	R_g
$x_{0,2}$	$15 \cdot R_g$
$y_{0,2}$	$2.5 \cdot R_g$
$v_{x0,2}$	$\frac{-R_g}{2 \cdot \tau}$
$v_{y0,2}$	0
4 - Low Mass and Head On Collision	
Galaxy 1	
N_1	1000
M_1	M_g
R_1	R_g
Offsets (position, velocity)	None
Galaxy 2	
N_2	500
M_2	$1.54 \cdot 10^{10} \cdot M_\odot$
R_2	$3.8 \cdot 10^{17}$
$x_{0,2}$	0
$y_{0,2}$	$2.5 \cdot R_g$
$v_{x0,2}$	$\frac{-R_g}{2 \cdot \tau}$
$v_{y0,2}$	0

Figure 1: Table summarising the parameters of the Galaxy Merger Simulations